AI Learning Assistant

Automated AI Research Curation and Learning

Technical Project Report

Platform:	n8n Cloud
Interface:	Telegram Bot API
Persistence:	Google Sheets
AI Integration:	OpenRouter
Project Level:	Level 3 — Advanced

GitHub Repository

https://github.com/camille-tanguy/ai-learning-assistant

October 2026

Abstract

The rapid growth of artificial intelligence research makes it increasingly difficult for students and practitioners to identify relevant publications efficiently.

This project presents an automated research curation and learning assistant developed using n8n Cloud, Telegram, Google Sheets, and OpenRouter.

The system retrieves research papers from arXiv and Hugging Face, normalizes and deduplicates records, applies deterministic relevance filtering, and uses large language models to evaluate selected publications.

A weighted ranking mechanism identifies up to five papers for a daily digest, while a Telegram interface supports on-demand retrieval and educational explanations.

An observed pipeline execution processed 1,403 retrieved records, selected 30 candidates for AI analysis, and produced a five-paper digest.

The architecture demonstrates how deterministic processing and targeted LLM integration can support cost-aware workflow automation.

Keywords: Workflow automation, n8n, LLM, data processing, research curation, Telegram, API integration.

1. Introduction

The rapid development of artificial intelligence has led to a growing volume of research publications distributed across platforms such as arXiv and Hugging Face.

Although these platforms provide access to recent developments, identifying relevant and educational contributions requires considerable time.

The AI Learning Assistant addresses this challenge by automating research discovery, evaluation, and organization.

The system combines conventional data-processing techniques with selective large language model (LLM) inference.

Its primary objectives are to retrieve publications from reliable sources, normalize heterogeneous records, eliminate duplicates, identify relevant research, and generate a curated daily digest.

Users can retrieve the digest and request deeper explanations of individual publications through Telegram.

A central design principle guides the implementation:

AI-powered, not AI-dependent.

Deterministic algorithms perform operations that do not require semantic reasoning, while LLMs are reserved for article evaluation and educational explanations.

The project targets Level 3 of the course's workflow automation framework through external API integration, persistent storage, branching logic, reusable sub-workflows, and AI processing.

2. System Architecture

2.1 Architectural Overview

The application is implemented as four separate n8n workflows, each responsible for a specific component of the system.

This modular architecture separates research ingestion, semantic analysis, persistent storage, and user interaction.

The workflows are:

1. **Telegram Connection:** Validates communication between n8n and Telegram.
2. **AI Daily Pipeline:** Retrieves, processes, evaluates, and stores publications.
3. **Article Analysis:** Performs structured LLM-based evaluation.
4. **Telegram Interaction:** Handles user commands and generates on-demand explanations.

The application supports manual execution, scheduled execution, and user-triggered interactions.

Google Sheets serves as the persistent data layer, while OpenRouter provides access to language models.

2.2 System Architecture Diagram

Figure 1 illustrates the main processing pipeline and its external integrations.

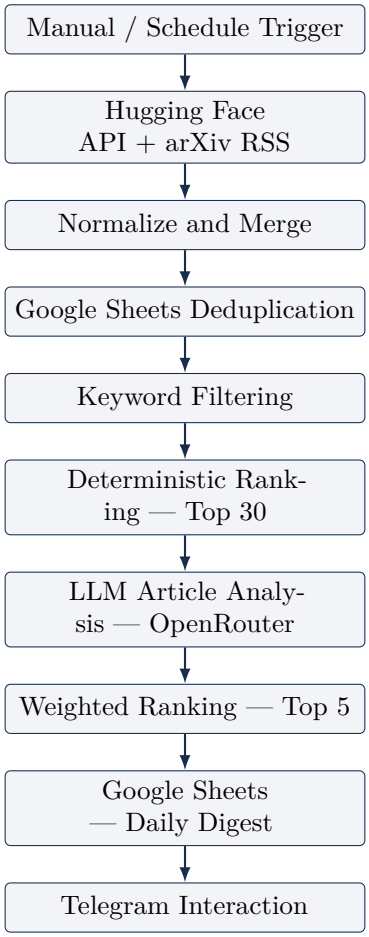


Figure 1: Main processing pipeline and external integrations. The scheduled pipeline stores the digest, which can subsequently be retrieved through Telegram.

2.3 Persistent Storage and User Interaction

Google Sheets provides persistent storage through two worksheets.

The `ARTICLES` worksheet stores publication identifiers, metadata, generated explanations, scores, and processing timestamps.

The `DAILY_DIGEST` worksheet stores selected article identifiers, formatted digest text, dates, and timestamps.

The Telegram interface supports four commands:

Command	Function	LLM
`/start`	Welcome and digest availability check	No
`/help`	Display instructions	No
`/daily`	Retrieve latest stored digest	No
`/learn [id]`	Generate a deeper explanation	Yes

Table 1: Implemented Telegram commands.

The scheduled pipeline generates and stores the daily digest without automatically delivering it through Telegram.

Users retrieve it through the interaction workflow. The `/start` branch can also initiate digest generation when no digest is available for the current day.

3. Data Collection and Processing

3.1 Research Data Collection and Normalization

The pipeline retrieves publications from two public sources: Hugging Face Daily Papers and the arXiv Computer Science — Artificial Intelligence RSS feed.

Both are accessed through n8n HTTP Request nodes.

Because the sources return different data structures, dedicated JavaScript Code nodes normalize their outputs.

The normalized representation includes an arXiv identifier, title, URL, publication date, source, description, and links to the original publication.

This common structure allows subsequent processing stages to operate independently of the original data source.

3.2 Deduplication

Duplicate publications may occur because Hugging Face frequently features papers originally published on arXiv.

The pipeline therefore implements deterministic deduplication using JavaScript `Set` and `Map` objects.

The algorithm first retrieves existing article records from Google Sheets.

Incoming records are then grouped using normalized arXiv identifiers.

When the same publication appears in both sources, the records are merged to preserve useful metadata and available reading links.

Finally, articles already stored in Google Sheets are excluded from further processing.

This mechanism prevents unnecessary repeated analysis and reduces the number of LLM requests.

3.3 Deterministic Relevance Filtering

After deduplication, publications are evaluated using a predefined set of AI-related keywords.

These include machine learning, deep learning, large language models, neural networks, computer vision, transformers, and robotics.

The filtering node combines the title and description, converts the resulting text to lowercase, and searches for matching expressions.

An article passes the filter when at least one keyword expression is detected.

The number of distinct matched expressions is recorded for preliminary ranking.

This method is inexpensive and transparent, although it cannot capture the full semantic meaning of a publication.

3.4 Preliminary Ranking

Relevant articles are assigned a deterministic score:

$$S_{\text{det}} = 2K + B \tag{1}$$

where K is the number of distinct matched keyword expressions and B is a content-length bonus.

The bonus is defined as:

$$B = \begin{cases} 1 & \text{if title and description exceed 800 characters,} \\ 0 & \text{otherwise.} \end{cases} \tag{2}$$

Articles are sorted by decreasing deterministic score.

Only the 30 highest-ranked candidates proceed to LLM-based analysis.

The purpose of this stage is to control inference volume rather than establish scientific research quality.

4. AI Integration and Ranking

4.1 LLM Integration

Semantic evaluation is performed through OpenRouter using its free-model routing configuration:

`openrouter/free`

The system uses LLM inference for two tasks: structured article evaluation and on-demand educational explanations.

The Article Analysis workflow receives publication metadata and requests a JSON response containing the article category, explanations, key takeaways, and three numerical scores.

The evaluation criteria are:

- **Relevance:** Importance for AI, machine learning, and software engineering.
- **Learning value:** Educational usefulness for the target reader.
- **Novelty:** Perceived distinctiveness of the contribution.

Each score ranges from 0 to 10.

4.2 Structured Output Validation

LLM-generated responses may contain invalid JSON, missing fields, or unexpected values.

The Article Analysis workflow therefore includes a JavaScript validation stage.

The node removes possible Markdown code fences, parses the response, verifies required fields, checks numerical score ranges, and confirms that explanations and key takeaways are present.

A Boolean field, `valid_json`, indicates whether the response satisfies the implemented structural requirements.

Invalid analyses are excluded from final ranking.

This improves data consistency but does not guarantee the factual correctness of explanations.

4.3 Final Ranking

After validation, the pipeline computes a weighted final score:

$$S_{\text{final}} = 0.40R + 0.35L + 0.25N \tag{3}$$

where:

- R represents relevance,

- L represents learning value,
- N represents novelty.

Relevance receives the highest weight, followed by learning value and novelty.

This reflects the objective of prioritizing publications that are both relevant and educational.

The five highest-scoring valid articles are selected for the daily digest.

The weights are design choices rather than empirically optimized parameters.

4.4 On-Demand Learning

The `/learn [article_id]` command retrieves a previously stored article from Google Sheets.

Its metadata and explanations are supplied to an LLM, which generates an educational response.

The response explains the central idea, the technical mechanism, its importance, and the concepts users can learn.

Unlike the daily analysis workflow, this component produces human-readable text rather than structured JSON.

5. Cost Optimization

5.1 Cost-Aware Design

Minimizing unnecessary LLM inference is a central objective of the project.

The application performs normalization, deduplication, keyword filtering, and preliminary ranking deterministically.

LLMs are invoked only after candidate selection or when a user requests a deeper explanation.

This reduces dependence on external inference services while maintaining semantic functionality.

5.2 Inference Volume Control

Let N represent retrieved records, D distinct new articles, R relevant articles, and A articles selected for analysis.

The pipeline satisfies:

$$A = \min(R, 30), \qquad A \leq R \leq D \leq N. \tag{4}$$

The fixed 30-candidate limit establishes an upper bound on normal article-analysis requests per execution.

Additional benefits arise from persistent deduplication and keyword filtering.

5.3 Remaining Costs

The application uses public research sources and OpenRouter's free-model routing to reduce external service costs.

However, free-model availability and request limits may vary.

n8n Cloud hosting may also introduce subscription costs independently of LLM usage.

Consequently, fewer inference requests should not be interpreted as equivalent to measured monetary savings.

6. Testing and Execution Results

6.1 Testing Approach

The application was tested through individual n8n workflow executions and Telegram interactions.

The evaluation focuses on article-processing counts, successful workflow completion, and the generation of the daily digest.

The results below correspond to observed executions rather than repeated benchmarking experiments.

6.2 Observed Processing Results

A successful AI Daily Pipeline execution on October 15, 2026 processed 1,403 retrieved article records.

After deduplication, 1,388 records remained.

The keyword filter retained 1,035 articles and rejected 353.

The deterministic ranking stage selected 30 candidates for semantic analysis.

Five papers were subsequently selected for the daily digest.

Processing Stage	**Records**
Retrieved records	1,403
Excluded by deduplication	15
Records after deduplication	1,388
Rejected by keyword filtering	353
Passed keyword filtering	1,035
Selected for LLM analysis	30
Selected for daily digest	5
Generated digest records	1

Table 2: Observed article-processing results from one successful pipeline execution.

6.3 Inference Volume Reduction

The observed processing counts allow the candidate reduction to be calculated.

Relative to analyzing every retrieved record:

$$\text{Reduction} = \frac{1403 - 30}{1403} \times 100 \approx 97.9\%. \tag{5}$$

The pipeline therefore selected approximately 97.9% fewer records for LLM analysis than a baseline analyzing every retrieved record.

This reduction reflects the combined effects of deduplication, relevance filtering, and the fixed Top 30 candidate limit.

It does not represent measured token savings, monetary savings, or an improvement in scientific recommendation quality.

6.4 Execution Performance

Execution history provided the following observed durations.

Workflow	Status	Duration
AI Daily Pipeline	Succeeded	12 min 55.148 s
Article Analysis	Succeeded	19.146 s
Telegram Interaction	Succeeded	2.292 s

Table 3: Observed execution durations. The Article Analysis result represents one sub-workflow execution, not the complete analysis batch.

The complete pipeline required approximately 12 minutes and 55 seconds.

This duration includes external requests, data transformations, Google Sheets operations, sequential analysis, and digest generation.

The Article Analysis and Telegram durations represent individual observed executions.

Additional observations would be necessary to estimate average performance, variability, or overall reliability.

6.5 Reliability Mechanisms

The application includes several mechanisms intended to improve reliability.

The external research retrieval nodes are configured with request retries.

The Article Analysis workflow validates the structure of generated JSON.

The Telegram interaction workflow checks for missing or unknown article identifiers.

Long digest messages are also split into smaller sections before delivery.

Nevertheless, these safeguards provide partial failure handling rather than guaranteed recovery from all external outages.

The system does not currently include a comprehensive automated regression testing or execution-monitoring framework.

7. Limitations and Future Improvements

The current implementation has several limitations.

First, research coverage is restricted to Hugging Face Daily Papers and arXiv cs.AI.

Second, keyword filtering may exclude relevant articles that use unfamiliar terminology.

Third, semantic evaluation relies primarily on publication titles and descriptions, rather than complete research papers.

Consequently, generated explanations and scores should be interpreted as preliminary learning-oriented assessments.

Fourth, Google Sheets provides simple persistence, but it is not designed for high-volume querying or concurrent application workloads.

Finally, the application lacks systematic monitoring of token usage, execution failures, and long-term reliability.

Future improvements should prioritize:

- Automated testing of data-processing components.
- Execution monitoring and token tracking.
- Automatic Telegram delivery of scheduled digests.

- Additional reliable research sources.
- User feedback through a `/like` command.
- Evaluation of LLM scores against human assessments.

These improvements should be introduced incrementally to preserve the project's low-cost and maintainable architecture.

8. Conclusion

The AI Learning Assistant demonstrates the implementation of a modular, AI-powered research curation system using n8n Cloud.

The application integrates public research sources, deterministic data processing, LLM-based evaluation, persistent storage, and a Telegram interface.

Its main technical contribution is the separation of conventional processing from tasks requiring semantic reasoning.

Deduplication, filtering, ranking, and command routing are implemented without unnecessary LLM inference.

An observed execution processed 1,403 retrieved records, selected 30 candidates for semantic analysis, and generated a five-paper digest.

These results demonstrate the operation of the processing pipeline and its ability to control inference volume.

However, systematic reliability measurements, factual evaluation of LLM outputs, and long-term cost monitoring remain necessary for a more comprehensive assessment.

Overall, the project illustrates how conventional software engineering practices and targeted AI integration can be combined to develop an understandable, maintainable, and cost-conscious learning assistant.